FASE 2 — DESARROLLO DEL AGENTE INTELIGENTE

TrafficFlow-Agent: Agente de Predicción y Control de Congestión Vehicular

- **Estudiante:** Jesús Andrés Pérez Fuentes (T00078491)
- **Curso:** Inteligencia Artificial — Universidad Tecnológica de Bolívar (UTB)
- **Docente:** Edwin Alexander Puertas Del Castillo
- **Fecha:** Octubre de 2026
- **Repositorio:** https://github.com/Jxzus/TRAFFICFLOW-AGENT

1. Correspondencia entre el Diseño (Fase 1) y la Implementación (Fase 2)

El agente implementado en esta Fase 2 corresponde de manera estricta al diseño formal presentado en el documento de Fase 1 (*Diseño Inicial de un Agente Inteligente*), manteniendo el mismo entorno, los mismos sensores, los mismos actuadores y la misma taxonomía según *Artificial Intelligence: A Modern Approach* (AIMA, Russell & Norvig).

| Dimensión | Especificación de Fase 1 (Diseño) | Implementación de Fase 2 (Código) | Estado / Justificación |

|---|---|---|---|

| **Tipo de Agente** | Agente basado en modelo con capacidad de aprendizaje (*Model-based Learning Agent*). | Clase `TrafficFlowAgent` en `src/trafficflow/agent.py`. Mantiene estado interno del corredor, buffer temporal, perfil histórico y disparador de aprendizaje adaptativo. | **Cumplido al 100%**. |

| **Entorno Operativo** | Corredor vial instrumentado con sensores fijos de velocidad a intervalos de 5 minutos. | Dataset benchmark `METR-LA` (207 sensores en Los Ángeles) y clase `TrafficEnvironment` en `src/trafficflow/environment.py`. | **Cumplido al 100%**. Incluye además generador sintético estocástico para pruebas en cualquier equipo. |

| **Sensores / Entradas (Sección 5)** | Velocidades promedio (mph/km/h), marcas temporales (reloj del sistema), historial de patrones típicos, detección de fallas de sensor (valores missing). | Método `perceive()` procesa lecturas continuas, enmascara ceros/NaNs y extrae variables de tiempo. | **Cumplido al 100%**. |

| **Actuadores / Salidas (Sección 6)** | Ajuste de planes semafóricos (`PICO_RECURRENTE`, `INCIDENTE`, `RESTABLECER_PLAN_BASE`) y mensajes a conductores/paneles variables. | Módulo `src/trafficflow/actuator.py` con interfaz `TrafficManagementClient` y actuador emulado `SimulatedTMS`. | **Cumplido al 100%**. |

| **Medida de Desempeño (SMART)** | Error Porcentual Absoluto Medio (MAPE) a 30 minutos inferior al **10 %** de forma continua. | Clase `ForecastTracker` calcula MAPE

móvil a 30 minutos y valida continuamente el cumplimiento de la meta. | **Cumplido al 100%**. |

Justificación de mejoras y adaptaciones técnicas realizadas:

1. **Descomposición explícita de métodos:** Para máxima claridad pedagógica y cumplimiento de la rúbrica, la función de transición del agente se separó explícitamente en cuatro métodos modulares: `perceive()`, `decide()`, `act()` y `learn()`.

2. **Abstracción del Entorno (`TrafficEnvironment`):** Se creó una clase formal de simulación de ambiente (`src/trafficflow/environment.py`) que implementa el ciclo AIMA (percepción/acción), permitiendo operar tanto en lazo abierto (*open-loop* sobre datos históricos) como en lazo cerrado (*closed-loop*, donde la extensión de verde semafórico mitiga la congestión en pasos posteriores).

3. **Capa de compatibilidad HDF5:** Se incorporó un mecanismo de lectura de doble vía en `data.py` (vía `pd.read_hdf` con respaldo nativo en `PyTables`) para garantizar compatibilidad con Python 3.12 y Pandas 3.x.

2. Función del Agente Explícita: Ciclo Percepción → Decisión → Acción → Aprendizaje

En el archivo [`src/trafficflow/agent.py`](file:///c:/Users/jesus/Universidad/Materias/2026-2/inteligencia%20artificial/proyecto%20agente/TRAFFICFLOW-AGENT-1/src/trafficflow/agent.py), la función del agente está formalizada de forma inequívoca en el método `step()`, el cual orquesta las cuatro fases del ciclo agéntico:

```python
def step(self, ts: pd.Timestamp, speeds: np.ndarray) -> Assessment | None:

# 1. PERCEPCIÓN: Ingesta, limpieza y actualización de memoria

perception = self.perceive(ts, speeds)

# 2. APRENDIZAJE Y CRÍTICA: Evaluación de predicciones pasadas y recalibración

learning_event = self.learn()

# 3. DECISIÓN: Pronóstico espacial-temporal y clasificación de estado

decision = self.decide(perception)

# 4. ACCIÓN: Envío de órdenes a controladores semafóricos y mensajería

action_result = self.act(decision)

# 5. REGISTRO: Log auditable con marca de tiempo
```

self._log_execution_step(perception, decision, action_result, learning_event)

return decision

```

### Detalle de cada fase en el código:

1. **`agent.perceive(ts, speeds)` (Líneas 100–136):**

- Recibe la lectura física o simulada de los sensores y la marca de tiempo.

- Trata los ceros como datos faltantes (*missing values* por falla de sensor o lazo inductivo), generando máscaras booleanas.

- Envía la observación real al componente de crítica (`ForecastTracker.observe`) para que evalúe pronósticos emitidos hace 30 minutos.

- Almacena las lecturas en la memoria de corto plazo (`buf_v`, `buf_t`, tamaño 12 = 1 hora) y en el búfer histórico de 7 días (`hist_v`, `hist_t`).

- Retorna un objeto tipado `Perception`.

2. **`agent.decide(perception)` (Líneas 138–200):**

- Si no hay suficiente contexto histórico (primeros 12 pasos), retorna `None` mientras acumula memoria.

- Ejecuta el modelo predictivo espacial-temporal (`GraphWaveNet` o líneas base) para estimar velocidades de 5 a 60 minutos en los 207 sensores.

- Registra el nuevo pronóstico en el rastreador (`ForecastTracker.register`).

- Calcula el nivel de congestión del corredor vial tomando el percentil 25 de la velocidad proyectada a 30 minutos contra los umbrales: $\ge 50$ mph (fluido), $\ge 35$ mph (moderado), $\ge 20$ mph (denso), $< 20$ mph (congestionado).

- Contrasta con el `HistoricalProfile` del corredor para ese día y franja horaria mediante el cálculo del $z\text{-score} = \frac{v_{\text{pred}} - \mu}{\sigma}$.

- Clasifica el estado:

- `"atipica"`: Si $\ge 15\%$ de los sensores tienen $z < -2.0$ y el corredor no está fluido (evidencia de accidente, obra o bloqueo imprevisto).

- `"recurrente"`: Si el nivel es denso o congestionado pero coherente con el patrón habitual de hora pico.

- `"normal"`: En cualquier otro caso.
```

- Selecciona el plan semafórico adecuado (`INCIDENTE` o `PICO_RECURRENTE`) y genera el mensaje de advertencia. Retorna `Assessment`.

3. **`agent.act(decision)` (Líneas 202–215):**

- Invoca el módulo actuador `apply_assessment()`.

- Incorpora una política de estabilización temporal e histéresis: solo transmite órdenes si el estado cambió o cada 6 pasos (30 minutos) para confirmación, evitando desincronización o fatiga en los controladores semafóricos.

- Envía comandos `adjust_signal_timings()` (extensión de fase verde de +15 % o +30 %) y `publish_driver_message()`. Retorna `ActionResult`.

4. **`agent.learn()` (Líneas 217–260):**

- Representa el elemento de aprendizaje de AIMA.

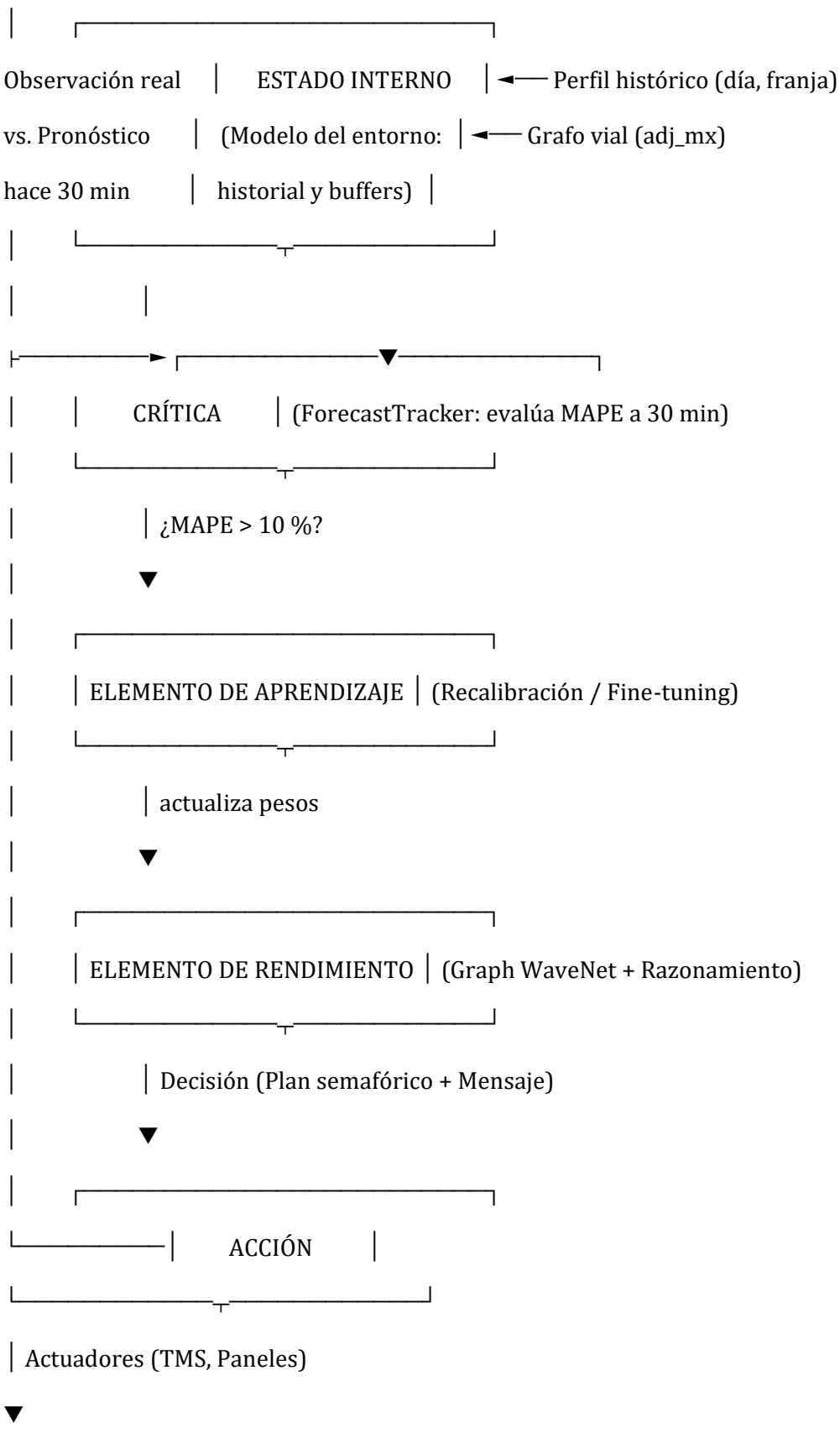
- Consulta el MAPE móvil a 30 minutos acumulado por el componente de crítica en la ventana reciente (24 horas).

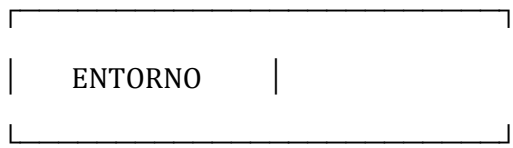
- Compara contra la meta SMART ($MAPE < 10\%$). Si se detecta deriva del modelo ($MAPE > 10\%$) y ha transcurrido el tiempo de enfriamiento (*cooldown* de 24 horas), dispara una recalibración (*fine-tuning*) del modelo utilizando los últimos 7 días de datos retenidos en memoria.

3. Coherencia con el Tipo de Agente (AIMA Cap. 2)

TrafficFlow-Agent es un **Agente Basado en Modelo con Capacidad de Aprendizaje**:

```
┌──────────────────────────────────┐
│      ENTORNO                     │
└──────────────┬───────────────────┘
               │ Sensores (5 min)
               ▼
┌──────────────────────────────────┐
┌─────────────►│      PERCEPCIÓN  │
│              └──────────────┬───┘
│                             │
│                             ▼
```


Observación real vs. Pronóstico hace 30 min
ESTADO INTERNO (Modelo del entorno: historial y buffers)
Perfil histórico (día, franja)
Grafo vial (adj_mx)
CRÍTICA (ForecastTracker: evalúa MAPE a 30 min)
¿MAPE > 10 %?
ELEMENTO DE APRENDIZAJE (Recalibración / Fine-tuning)
actualiza pesos
ELEMENTO DE RENDIMIENTO (Graph WaveNet + Razonamiento)
Decisión (Plan semafórico + Mensaje)
ACCIÓN
Actuadores (TMS, Paneles)




```

1. **Mantenimiento de Estado Interno:** El entorno vial no es completamente observable en un solo instante de tiempo; la velocidad instantánea no revela si una lentitud es normal o anómala, ni hacia dónde se propagará la onda de choque vehicular. Por ello, el agente almacena:

- Buffer de velocidades y marcas temporales de la última hora.

- Estructura de conectividad vial (matriz de adyacencia del grafo).

- Memoria estadística de largo plazo (media $\mu$ y desviación $\sigma$ por sensor y franja).

- Cola de pronósticos pendientes para su cotejo cronológico posterior.

2. **Elemento de Crítica:** `ForecastTracker` compara de manera desacoplada en el tiempo el pronóstico emitido en $t - 6$ con la observación real en $t$, calculando de forma continua el error relativo absoluto.

3. **Elemento de Aprendizaje:** Cuando la crítica señala que el desempeño se degrada ($MAPE > 10\%$), el agente no continúa con parámetros estáticos; ajusta su modelo predictivo con la experiencia reciente.

4. **Elemento de Rendimiento:** Genera las predicciones y mapea el estado proyectado hacia la acción semafórica de mayor utilidad.

## 4. Entorno Real y Simulado

El sistema soporta indistintamente el entorno real y un entorno simulado estocástico mediante la clase [`TrafficEnvironment`](file:///c:/Users/jesus/Universidad/Materias/2026-2/inteligencia%20artificial/proyecto%20agente/TRAFFICFLOW-AGENT-1/src/trafficflow/environment.py):

### Características de Ambiente (Sección 3 del Diseño) implementadas en el Simulador:

1. **Parcialmente Observable:** Se introducen tasas de falla aleatoria de sensores (`sensor_failure_prob`, con valores $0.0$ que simulan desconexión o falla de lazo inductivo), forzando al agente a lidiar con observaciones incompletas e imputaciones enmascaradas.

2. **Estocástico:** El simulador inyecta ruido gaussiano en las velocidades (`stochastic_noise_std`) y eventos imprevistos (caídas severas de velocidad de duración y
```

ubicación aleatoria en `synthetic.py`), asegurando que transiciones idénticas no conduzcan siempre al mismo resultado determinista.

3. **Secuencial:** Cada estado de tráfico condiciona el siguiente. La simulación emula la propagación espacial de la onda de congestión (cada sensor transmite parte del retraso a los sensores contiguos: `speeds[1:, j] -= 0.15 * (base - speeds[:-1, j - 1])`).

4. **Dinámico:** El entorno avanza en intervalos estrictos de 5 minutos; el estado del tráfico cambia independientemente del tiempo que tarde el agente en procesar.

5. **Continuo:** La velocidad física es una variable continua expresada en mph, discretizada temporalmente para la ingesta.

6. **Agente único en entorno multiagente:** El agente interactúa de forma centralizada con la red de semáforos, mientras el flujo vehicular de miles de conductores reacciona de forma distribuida.

Modo Lazo Cerrado (*Closed-Loop*):

En `TrafficEnvironment`, al habilitar `--closed-loop`, el simulador aplica el efecto físico del actuador: si el agente despliega `PICO_RECURRENTE` o `INCIDENTE`, la extensión del verde semafórico alivia el cuello de botella aumentando la velocidad de los sensores afectados entre 2.5 y 4.0 mph, permitiendo evaluar la efectividad de las decisiones del agente.

5. Registro (Log) de Ejecución Auditable

Tal como lo exige el enunciado para constituir la base de las pruebas experimentales de la **Fase 3**, cada ejecución genera automáticamente el archivo [`outputs/execution_log.jsonl`](file:///c:/Users/jesus/Universidad/Materias/2026-2/inteligencia%20artificial/proyecto%20agente/TRAFFICFLOW-AGENT-1/outputs/execution_log.jsonl), donde cada línea registra el ciclo íntegro del agente:

```json

{

"step": 23,

"timestamp": "2012-06-04 06:35:00",

"perception": {

"timestamp": "2012-06-04 06:35:00",

"num_sensors": 207,

"valid_sensors": 206,

"missing_sensors": 1,
```

"mean_speed_observed_mph": 59.16,

"min_speed_observed_mph": 12.63

},

"decision": {

"timestamp": "2012-06-04 06:35:00",

"status": "normal",

"level": 0,

"level_name": "fluido",

"corridor_speed_mph": 57.39,

"atypical_fraction": 0.0048,

"affected_sensors": []

},

"action": {

"action_taken": false,

"plan": null,

"green_extension_pct": null,

"message": null,

"sensors": [],

"timestamp": "2012-06-04 06:35:00"

},

"learning": {

"step": 23,

"rolling_mape_30min": 0.0492,

"samples": 7,

"meets_target": true,

"recalibrated": false

}

}

```

Adicionalmente, se generan:

- `outputs/tms_actions.jsonl`: Bitácora específica de eventos transmitidos al sistema de gestión de tráfico.
- `outputs/simulation_report.json`: Resumen estadístico integral (MAE/MAPE por horizonte temporal, total de acciones, desglose de estados y cumplimiento de la meta SMART).

## 6. Tecnologías y Atribución del Razonamiento

De acuerdo con el requisito de transparencia tecnológica, se detalla qué parte del razonamiento ejecuta el código desarrollado y qué parte delega en bibliotecas externas:

| Componente | Responsable | Descripción y Justificación Técnica |

|---|---|---|

| **Ciclo Agéntico y Orquestación** | Código propio (`agent.py`, `environment.py`) | Lógica de control de estados, orquestación del ciclo percepción $\to$ decisión $\to$ acción $\to$ aprendizaje, manejo de histéresis y persistencia de logs. |

| **Detección de Atípicos vs. Recurrentes** | Código propio (`agent.py`, `profile.py`) | Modelado estadístico con cálculo de $z$-scores espaciales contra perfiles típicos históricos, evaluación del percentil 25 del corredor y categorización de incidentes. |

| **Política de Selección de Acciones** | Código propio (`actuator.py`) | Reglas de asignación de planes semafóricos (`PICO_RECURRENTE`, `INCIDENTE`, `RESTABLECER_PLAN_BASE`) y supresión de oscilaciones transitorias (*cooling*). |

| **Componente de Crítica y Deriva** | Código propio (`agent.py`: `ForecastTracker`) | Seguimiento temporal asíncrono entre pronósticos pasados y observaciones reales, cálculo de error móvil y activación de recalibración. |

| **Red Espacial-Temporal (Graph WaveNet)** | PyTorch (`model.py`) | Operaciones tensoriales de bajo nivel: convoluciones causales dilatadas temporales (TCN) combinadas con capas de difusión sobre el grafo vial con adyacencia adaptativa autoaprendida. |

| **Estructuras de Datos y Series Temporales** | NumPy / Pandas (`data.py`) | Manipulación de matrices tensoriales, alineación de índices temporales de 5 minutos y cálculo de percentiles y medias enmascaradas. |

| **Almacenamiento Estructurado HDF5** | PyTables / HDFStore | Lectura eficiente de los 34.272 pasos temporales del dataset real `METR-LA`. |
```

> **Nota sobre Modelos de Lenguaje (LLMs):** En concordancia con los requisitos de tiempo real (latencia estricta $< 5\text{ segundos}$ en ciclos de 5 min) y la naturaleza espacial continua de los sensores viales, el razonamiento central del agente se ejecuta mediante el modelo de aprendizaje profundo espacial-temporal Graph WaveNet y las políticas deterministas de decisión basadas en teoría de tráfico. No se empleó un LLM externo en el bucle cerrado de control para evitar alucinaciones en planes de control semafórico crítico y latencias de red en la infraestructura vial.

7. Instrucciones de Reproducibilidad y Ejecución

El código es 100 % reproducible en cualquier equipo con Python 3.10, 3.11 o 3.12 (Windows, macOS o Linux).

Paso 1: Clonar y crear entorno virtual

```bash
git clone https://github.com/Jxzus/TRAFFICFLOW-AGENT.git

cd TRAFFICFLOW-AGENT-1

python -m venv .venv
```

En Windows:

```
.venv\Scripts\activate
```

En Linux / macOS:

source .venv/bin/activate

```

### Paso 2: Instalar dependencias

```bash
pip install -r requirements.txt

pip install -e ".[dev]"
```

Paso 3: Ejecutar pruebas automatizadas

Para verificar la integridad del ciclo del agente, el modelo, las métricas y el entorno:

```bash
pytest -v
```
```

```

*Resultado esperado: 8 pruebas aprobadas (100 % PASSED).*

### Paso 4: Ejecutar la simulación del agente en tiempo real

**Con datos reales (`METR-LA`):**

```bash

python -m trafficflow.simulate --steps 500 --out outputs

```

Con entorno sintético estocástico (sin necesidad de dataset externo):

```bash

python -m trafficflow.simulate --synthetic --noise-std 2.0 --failure-prob 0.05 --closed-loop --steps 500 --out outputs

```

Verificar los logs generados:

- Log completo con ciclo y marcas de tiempo: `outputs/execution_log.jsonl`
- Reporte global de desempeño SMART: `outputs/simulation_report.json`
- Bitácora de acciones del centro de control: `outputs/tms_actions.jsonl`
```